

3D Art Gallery — Starter Code UAS

Starter project galeri seni 3D pakai **Three.js + Vite**. Sudah jalan penuh: karakter bisa jalan-jalan (WASD), klik lukisan untuk zoom ke karya, patung di tengah ruangan, lantai/tembok/atap bertekstur.

Tugas kalian: kembangkan project ini (ganti karya seni, karakter, tata ruang, tambah interaksi, dll — lihat instruksi UAS).

 **Baca** `MANUAL.md` untuk panduan lengkap: penjelasan tiap script, prosedur kustomisasi (lukisan/karakter/tembok), pengenalan agentic coding (Claude Code / Codex) + contoh prompt, dan cara deploy.

Cara menjalankan

Butuh **Node.js** (<https://nodejs.org>) dan editor (mis. VS Code).

```
npm install      # sekali saja, install dependency
npm run dev      # jalankan dev server → buka http://localhost:5173
```

`npm run dev` auto-reload tiap kalian save file.

Kontrol

Tombol	Aksi
W A S D	Gerak karakter
Mouse	Lihat sekeliling
Space	Toggle pointer lock
Klik lukisan	Zoom ke karya (klik lagi / WASD / Esc untuk keluar)
M	Menu • Enter

Struktur project

```
index.html      # halaman utama
main.js         # entry point
style.css       # styling UI
modules/        # semua logika scene (dipecah per fitur)
├─ paintings.js / paintingData.js # lukisan & datanya
├─ paintingFocus.js               # zoom saat lukisan diklik
├─ player.js / movement.js       # karakter & gerakan
├─ statue.js                     # patung tengah ruangan
├─ floor.js / walls.js / ceiling.js
├─ lighting.js                   # pencahayaan
└─ ...
public/         # aset statis (gambar, model 3D .glb/.gltf, suara)
├─ artworks/    # gambar lukisan (1.jpg, 2.jpg, ...)
├─ models/      # model 3D
└─ img/         # tekstur
```

Yang sering diubah mahasiswa

- **Ganti lukisan** → taruh gambar di `public/artworks/` dan edit `modules/paintingData.js` (judul, artist, deskripsi, posisi di tembok).
- **Ganti karakter** → ganti file `.glb` di `public/models/guide/` dan sesuaikan konstanta di atas `modules/player.js` (scale, offset).
- **Ganti patung** → ganti `.glb` di `public/models/statue/` dan path di `modules/statue.js`.
- **Ganti tekstur lantai/tembok** → edit `modules/floor.js` / `walls.js`.

⚠ Kalau memuat aset baru dengan path absolut (`loader.load("/...")`), gunakan prefix `import.meta.env.BASE_URL` (lihat contoh di `floor.js` / `statue.js`) supaya tetap jalan saat di-deploy ke GitHub Pages.

Deploy ke GitHub Pages (opsional)

1. Di `vite.config.js`, set `base: "/nama-repo-kalian/"`.
2. `npm run build` → menghasilkan folder `dist/`.
3. Push isi `dist/` ke branch `gh-pages` repo kalian, lalu aktifkan **Settings** → **Pages** → **branch** `gh-pages`.

Berbasis tutorial "3D Art Gallery using Three.js" oleh Emilian Kasemi
(<https://github.com/theringsofsaturn/3D-art-gallery>). Lisensi: CC BY-NC-SA 4.0.

MANUAL & PANDUAN UAS — 3D Art Gallery (Three.js)

Dokumen ini panduan lengkap untuk mengerjakan UAS memakai **starter code 3D Art Gallery**. Isi: penjelasan kode, fungsi tiap script, struktur aset/folder, prosedur kustomisasi manual (lukisan, karakter, tembok), pengenalan *agentic coding* (Claude Code / Codex), contoh prompt, dan cara deploy ke GitHub Pages.

Baca juga `README.md` untuk quick-start singkat.

Daftar Isi

1. [Tentang Project & Sitasi](#)
 2. [Menjalankan Project](#)
 3. [Arsitektur & Fungsi Tiap Script](#)
 4. [Struktur Folder & Aset](#)
 5. [Kustomisasi Manual](#)
 - 5.1 Ganti lukisan dengan portofolio + caption
 - 5.2 Ganti karakter dengan foto sendiri (Avaturn) + proses Mixamo
 - 5.3 Custom tembok dengan aset eksternal (motif Papua)
 6. [Agentic Coding: Konsep & Praktik](#)
 - 6.1 Apa itu agentic / vibe coding
 - 6.2 Konsep inti: context, scaffolding, hooks, skills/commands, MCP
 - 6.3 Setup Claude Code / Codex
 - 6.4 Contoh prompt (kustomisasi & perbaikan bug)
 7. [Deploy ke GitHub Pages](#)
 8. [Referensi](#)
-

1. Tentang Project & Sitasi

Galeri seni 3D interaktif berbasis **Three.js** + **Vite**. Pengunjung mengontrol karakter (third-person), berjalan keliling ruangan, melihat lukisan, dan mengklik lukisan untuk zoom ke karya.

Project ini diturunkan dari / mengacu pada:

- Repository asli: **theringsofsaturn / 3D-art-gallery-threejs** → <https://github.com/theringsofsaturn/3D-art-gallery-threejs>

- Tutorial **freeCodeCamp** (YouTube): "Build a 3D Art Gallery with Three.js" → <https://www.youtube.com/watch?v=imqiYwidUIA&t=4859s>
- Lisensi konten asli: Creative Commons **CC BY-NC-SA 4.0** (oleh Emilian Kasemi).

Starter ini sudah ditambah: karakter ber-animasi (idle/walk), interaksi klik lukisan → zoom POV, patung GLB di tengah ruangan, dan tekstur lantai dari file.

Wajib: cantumkan sitasi di atas pada laporan UAS kalian.

2. Menjalankan Project

Butuh **Node.js** (<https://nodejs.org>) + editor (mis. **VS Code**).

```
npm install      # install dependency (sekali saja)
npm run dev      # jalankan → buka http://localhost:5173
```

Kontrol:

Tombol	Aksi
W A S D	Gerak karakter
Mouse	Putar kamera
Space	Toggle pointer lock
Klik lukisan	Zoom ke karya (klik lagi / WASD / Esc = keluar)
Enter	Mulai jelajah • Esc stop • M menu • G/P audio

3. Arsitektur & Fungsi Tiap Script

Alur: `index.html` memuat `main.js` (entry point). `main.js` merakit semua modul: membuat scene → menambah objek (tembok, lantai, lukisan, karakter, patung) → memasang event listener & klik → menjalankan render loop.

Semua logika dipecah per fitur di folder `modules/` :

Script	Fungsi
<code>main.js</code> (root)	Entry point. Memanggil semua modul, merakit scene, mulai render loop.
<code>scene.js</code>	Membuat <code>Scene</code> , <code>Camera</code> , <code>Renderer</code> , dan <code>PointerLockControls</code> .

Script	Fungsi
sceneHelpers.js	Helper <code>addObjectsToScene()</code> untuk menambah banyak objek sekaligus.
walls.js	Membuat 4 tembok (Box) + tekstur.
floor.js	Membuat lantai + tekstur (dari file <code>public/img/Floor.jpg</code>).
ceiling.js	Membuat langit-langit + tekstur.
ceilingLamp.js	Memuat model lampu (<code>.gltf</code>) dan menaruhnya di langit-langit.
lighting.js	Cahaya: 1 <code>AmbientLight</code> + beberapa <code>SpotLight</code> (tembok & patung).
paintings.js	Membuat mesh lukisan (PlaneGeometry + tekstur gambar) dari data.
paintingData.js	Data lukisan: gambar, ukuran, posisi di tembok, judul, artist, deskripsi.
paintingInfo.js	Menampilkan/menyembunyikan panel caption (judul, artist, dll).
paintingFocus.js	Saat lukisan diklik: meluncurkan kamera ke POV depan karya.
clickHandling.js	Raycast dari klik mouse → deteksi lukisan → panggil focus.
player.js	Memuat karakter GLB + animasi idle/walk; ada fallback prosedural.
movement.js	Gerak WASD, kamera third-person, deteksi tabrakan , ayunan animasi.
boundingBox.js	Membuat <code>Box3</code> (kotak tabrakan) untuk tembok & lukisan .
statue.js	Memuat patung GLB di tengah ruangan + atur material/skala.
bench.js	Memuat model bangku (<code>.gltf</code>).
eventListeners.js	Menangani keyboard/mouse: pointer lock, menu, audio, exit zoom.
menu.js	Tombol Play & overlay menu.
audioGuide.js	Audio guide (musik/narasi).
rendering.js	Render loop (<code>requestAnimationFrame</code>): update gerak/zoom, gambar frame.
VRSupport.js	Tombol & dukungan WebXR/VR.
proceduralAssets.js	Membuat tekstur & geometri cadangan via canvas (fallback).

Catatan teknis penting (akan dipakai di bagian agentic coding):

- Tabrakan dihitung di `movement.js` → `checkCollision()` . Fungsi ini **hanya** mengecek `walls.children` . Lukisan & patung **tidak** dicek → karakter bisa menembusnya.
- Aset yang dimuat saat runtime dengan path absolut (`loader.load("/...")`) harus diberi prefix `import.meta.env.BASE_URL` agar tetap jalan ketika di-deploy ke subpath GitHub Pages (lihat contoh di `floor.js` , `statue.js` , `player.js`).

4. Struktur Folder & Aset

```
3d-gallery-starter/
├─ index.html          # halaman utama
├─ main.js             # entry point
├─ style.css           # styling UI (panel caption, menu, dll)
├─ vite.config.js      # konfigurasi Vite (base path untuk deploy)
├─ package.json        # dependency & script (dev/build)
├─ modules/            # semua logika scene (lihat tabel di atas)
└─ public/             # ASET STATIS (disalin apa adanya saat build)
    ├─ artworks/       # gambar lukisan: 1.jpg, 2.jpg, ... 16.jpg
    ├─ img/            # tekstur (mis. Floor.jpg)
    ├─ models/         # model 3D
    │   ├─ guide/      # karakter (character.glb = idle, walk.glb)
    │   ├─ statue/     # patung (aphrodite.glb)
    │   ├─ bench/ bench_2/ # bangku
    │   └─ ceiling-lamp/ # lampu
    └─ sounds/         # audio guide
```

Aturan emas aset: apa pun yang harus bisa diakses lewat URL (gambar, model, suara) **taruh di public/** . Path-nya jadi relatif terhadap **public/** . Contoh: `public/artworks/1.jpg` → dipanggil sebagai `artworks/1.jpg` .

5. Kustomisasi Manual

Bagian ini **tanpa AI** — supaya kalian paham dulu cara kerjanya. (Versi pakai AI ada di Bagian 6.)

5.1 Ganti lukisan dengan portofolio + caption

1. Siapkan gambar karya kalian (JPG/PNG, idealnya rasio ~5:3 mengikuti `width:5, height:3`).
2. Salin ke folder `public/artworks/` dan beri nama angka: `1.jpg` , `2.jpg` , dst. (timpa file contoh, atau pakai nama baru lalu sesuaikan langkah 3).
3. Buka `modules/paintingData.js` . Tiap lukisan adalah satu objek. Edit bagian `info` :

```
info: {
  title: "Judul Karya Saya",
  artist: "Nama Kalian / NIM",
  description: "Penjelasan singkat tentang karya ini.",
  year: "2026",
  link: "https://instagram.com/akun-kalian",
},
```

4. Untuk **memindah posisi** lukisan di tembok, ubah `position: { x, y, z }` dan `rotationY`. Pola yang sudah ada: tembok depan `z: -19.5`, belakang `z: 19.5` (`rotationY: Math.PI`), kiri `x: -19.5` (`Math.PI/2`), kanan `x: 19.5` (`-Math.PI/2`).
5. Simpan → browser auto-reload. Dekati lukisan / klik → caption muncul.

Saat ini data dibuat otomatis dengan `Array.from(...)` (4 lukisan per tembok). Kalau bingung, kalian boleh **menulis ulang** array `paintingData` secara manual (satu objek per lukisan) supaya lebih mudah diedit per karya.

5.2 Ganti karakter dengan foto sendiri (Avaturn) + proses Mixamo

Karakter ada di `public/models/guide/`:

- `character.glb` → model + animasi **idle** (diam).
- `walk.glb` → animasi **jalan**.

Membuat avatar dari foto (Avaturn):

1. Buka <https://avaturn.me>, upload foto wajah → generate avatar.
2. Export sebagai **GLB**.
3. (*Opsional, untuk animasi*) avatar Avaturn bisa dipasang animasi via Mixamo (lihat di bawah). Untuk UAS, **boleh pakai animasi idle/walk yang sudah ada** — syaratnya rangka (skeleton) tulangnya kompatibel.

Cara termudah (pakai animasi bawaan starter):

- Kalau hanya ingin ganti wajah/baju dan tetap pakai gerak yang ada, ganti file `character.glb` (dan `walk.glb`) dengan model kalian yang **sudah ber-skeleton sama**.
- Lalu sesuaikan konstanta di atas `modules/player.js`: `MODEL_SCALE` (ukuran), `MODEL_Y_OFFSET` (tinggi/agar tidak melayang), `MODEL_FACE_OFFSET` (set `Math.PI` kalau karakter menghadap terbalik).

Proses Mixamo (cara RESMI Avaturn — PENTING):

⚠ **Mixamo TIDAK bisa baca GLB**, dan dialog Download Avaturn **hanya menyediakan GLB** (Avatar T-Pose / Body only / Avatar with animation). Jadi FBX-nya **BUKAN** dari tombol Download. Kalian **tidak me-rig avatar sendiri** di Mixamo — Avaturn menyediakan **satu FBX generik** (skeleton-nya seragam dengan SEMUA avatar Avaturn) yang dipakai cuma untuk "memancing" animasi dari Mixamo. Animasi itu lalu ditransfer ke avatar GLB kalian di Blender. Panduan resmi: <https://docs.avaturn.me/docs/importing/mixamo/>

Langkah 1 — Ambil 2 file dari Avaturn

- Klik **Avatar (T-Pose)** → Download → ini **GLB avatar asli kalian** (wajah/baju kalian).
- Buka tutorial "**Use with Mixamo animations**" (di dialog Download) → unduh **FBX generik**.
Link langsung: https://assets.avaturn.me/mesh_for_mixamo_72940d11f8.fbx (Inilah `mesh_for_mixamo_*.fbx` — model standar, sama untuk semua orang.)

Langkah 2 — Ambil animasi di Mixamo

1. Buka <https://www.mixamo.com> (login akun Adobe gratis).
2. **Upload Character** → pilih `mesh_for_mixamo_*.fbx` (FBX generik tadi) → auto-rig.
3. Cari & pilih animasi "**Idle**". Download: format **FBX, Without Skin**, 30 fps.
4. Ulangi untuk "**Walking**". Download: **FBX, Without Skin**.

Langkah 3 — Transfer animasi ke avatar GLB kalian (Blender)

Karena skeleton FBX generik = skeleton avatar GLB kalian (nama tulang sama), action animasi bisa langsung "dipinjamkan". Lakukan **dua kali**: sekali untuk idle, sekali untuk walk.

1. **File** → **Import** → **glTF 2.0** → pilih **avatar GLB kalian** (muncul mesh + 1 armature).
2. **File** → **Import** → **FBX** → pilih **FBX Idle dari Mixamo** (muncul armature ke-2 + Action).
3. Ubah salah satu panel jadi **Dope Sheet**, lalu mode **Action Editor**.
4. Di viewport/Outliner, **klik armature AVATAR kalian** (yang dari GLB).
5. Di Action Editor, klik dropdown **Browse Action** (ikon kotak) → pilih action Mixamo (biasanya bernama `mixamo.com / Armature|mixamo.com`). Animasi langsung jalan di avatar kalian.
6. **Hapus** armature + mesh generik dari Mixamo (sisakan hanya avatar kalian).
7. **File** → **Export** → **glTF 2.0 (.glb)** → di panel kanan centang **Animation** → simpan sebagai `character.glb` (ini untuk idle).
8. **Ulangi langkah 1–7** dengan **FBX Walking** → export sebagai `walk.glb`.

Langkah 4 — Pasang di project

- Taruh `character.glb` & `walk.glb` di `public/models/guide/`.
- Sesuaikan `MODEL_SCALE` / `MODEL_Y_OFFSET` / `MODEL_FACE_OFFSET` di `modules/player.js`.

Alternatif Langkah 3 — TANPA buka Blender (mode command/headless)

Langkah 3 (transfer animasi) bisa dijalankan otomatis lewat **Blender headless** — tidak perlu buka GUI Blender sama sekali. Project ini menyediakan script `tools/anim_transfer.py`. Setelah punya avatar GLB + FBX animasi Mixamo, jalankan:

```
# Idle → character.glb
"C:\Program Files\Blender Foundation\Blender 5.1\blender.exe" --background \
  --python tools/anim_transfer.py -- avatar.glb idle.fbx public/models/guide/character.glb

# Walk → walk.glb
"C:\Program Files\Blender Foundation\Blender 5.1\blender.exe" --background \
  --python tools/anim_transfer.py -- avatar.glb walking.fbx public/models/guide/walk.glb
```

Ganti `avatar.glb`, `idle.fbx`, `walking.fbx` dengan path file kalian, dan sesuaikan path `blender.exe` dengan versi Blender yang terpasang. Script otomatis: import avatar + FBX animasi, samakan nama tulang (buang prefix `mixamorig:`), tempel animasi ke armature avatar, lalu export GLB. *(Ini adalah cara yang dipakai saat membuat starter ini.)*

Ringkasan file:

Dari mana	File	Fungsi
Tombol Avatar (T-Pose)	GLB	avatar asli kalian (di-import ke Blender)
Tutorial Use with Mixamo animations	<code>mesh_for_mixamo_*.fbx</code>	di-upload ke Mixamo untuk ambil animasi
Mixamo	FBX Idle / Walking	animasi → ditransfer ke GLB di Blender → export GLB

Tips bila action tidak muncul / hasil aneh: pastikan FBX Mixamo "Without Skin", dan kalau dropdown action kosong, cek di **Outliner** apakah FBX-nya benar ter-import. Jika tekstur avatar hilang (jadi abu-abu) setelah export, lihat catatan re-skin di bawah.

Catatan: kalau tekstur hilang (karakter jadi abu-abu) setelah proses ini, itu masalah umum — perlu *re-skin* mesh bertekstur ke armature animasi. Proses lanjutan; boleh dilewati untuk UAS atau minta bantuan AI (Bagian 6).

5.3 Custom tembok dengan aset eksternal (motif Papua)

Saat ini tembok memakai tekstur prosedural (`createWallTexture()` di `walls.js`). Untuk memakai gambar motif (mis. **motif Papua / Asmat**):

1. Siapkan file gambar motif (mis. `papua.jpg`) → taruh di `public/img/papua.jpg` . (Cari aset bebas-pakai / buat sendiri; cantumkan sumbernya di laporan.)

2. Buka `modules/walls.js` . Di paling atas, hapus import prosedural lalu pakai `TextureLoader` .
Contoh perubahan:

```
import * as THREE from "three";
// import { createWallTexture } from "../proceduralAssets.js"; // <- tidak dipakai

export function createWalls(scene) {
  // ...
  const wallTexture = new THREE.TextureLoader().load(
    `${import.meta.env.BASE_URL}img/papua.jpg`
  );
  wallTexture.wrapS = wallTexture.wrapT = THREE.RepeatWrapping;
  wallTexture.repeat.set(4, 2); // atur perulangan motif

  const wallMaterial = new THREE.MeshStandardMaterial({
    map: wallTexture,
    roughness: 0.85,
    metalness: 0,
    side: THREE.DoubleSide,
  });
  // ... sisanya tetap
}
```

3. Catatan: prefix `import.meta.env.BASE_URL` **wajib** supaya tetap muncul saat deploy.
4. Kalau motif terlihat melar/terlalu kecil, atur `repeat.set(x, y)` .

Mau hanya satu tembok yang bermotif (sisanya polos)? Buat dua material berbeda dan pasang material motif hanya pada mesh tembok yang diinginkan.

6. Agentic Coding: Konsep & Praktik

Setelah paham cara manual, kalian boleh mempercepat kustomisasi dengan **AI coding agent** seperti **Claude Code** atau **OpenAI Codex**. Pelajari dulu konsepnya.

Sumber belajar (Coursera):

- *Claude Code in Action* — modul: (1) What is Claude Code, (2) Getting Hands-On [setup, adding context, making changes, custom commands, MCP, GitHub integration], (3) Hooks & the SDK [introducing/defining/implementing hooks, useful hooks], (4) Wrap-up.

- *Introduction to OpenAI Codex* — pelajaran: intro, Codex Cloud setup, reviewing changes locally, analysis tasks, code reviews, **Codex CLI**, built-in commands, IDE extension, context/reasoning levels & todos, tokens & MCP, setting up MCP servers.

6.1 Apa itu agentic / vibe coding

- **Coding assistant biasa**: melengkapi kode baris-per-baris (autocomplete).
- **Agentic coding**: kalian memberi **tujuan dalam bahasa natural**, lalu agen **merencanakan, membaca file, mengubah banyak file, menjalankan perintah, dan memverifikasi** sendiri — kalian me-review hasilnya. (Mis. "ganti semua caption lukisan jadi karya saya").
- **Vibe coding**: gaya kerja di mana kalian fokus pada *maksud/hasil* dan membiarkan agen menangani detail implementasi — sangat cocok untuk eksplorasi & prototipe. ⚠ Tetap **review** tiap perubahan; jangan menerima buta.

6.2 Konsep inti

- **Context (konteks)** — agen bekerja lebih baik bila tahu soal project. Kalian bisa menaruh instruksi project di file `CLAUDE.md` (Claude Code) atau `AGENTS.md` (Codex) di root project: jelaskan struktur, perintah `npm run dev`, gaya kode, dll. (*Course 1* → "Adding context"; *Course 2* → "context, reasoning levels & todos".)
- **Scaffolding** — agen membuat *kerangka* awal (file/folder/boilerplate) sehingga kalian tinggal mengisi bagian penting. Hemat waktu setup.
- **Hooks** — skrip yang otomatis jalan **sebelum/sesudah** aksi agen (mis. jalankan formatter atau cek lint setelah file diedit, atau blokir edit file tertentu). (*Course 1* → modul "Hooks & the SDK".)
- **Skills / Custom commands** — perintah yang bisa dipakai ulang (mis. `/deploy`), membungkus instruksi panjang jadi satu shortcut. (*Course 1* → "Custom commands"; *Course 2* → "built-in commands in the Codex CLI".)
- **MCP (Model Context Protocol)** — cara menyambungkan agen ke alat/data eksternal (GitHub, database, dll). (*Kedua course punya bagian MCP.*)
- **Review changes** — selalu periksa diff sebelum commit. (*Course 2* → "reviewing changes locally", "Codex code reviews".)

6.3 Setup Claude Code / Codex

- **Claude Code (CLI)**: install (`npm i -g @anthropic-ai/claude-code`), jalankan `claude` di dalam folder project, login. Lihat *Course 1* → "Claude Code setup".
- **OpenAI Codex (CLI)**: install Codex CLI, login, jalankan di folder project. Lihat *Course 2* → "Codex CLI" & "the Codex IDE extension".
- **Tips**: buka agen **di dalam folder** `3d-gallery-starter` supaya ia bisa membaca semua file project sebagai konteks.

6.4 Contoh prompt

Tulis prompt yang **spesifik** dan sebutkan **file/letaknya** bila tahu. Contoh:

A. Kustomisasi caption & lukisan

Di `modules/paintingData.js`, ganti semua lukisan jadi 6 karya portofolio saya. Gambarnya sudah saya taruh di `public/artworks/` (`a1.jpg` s/d `a6.jpg`). Untuk tiap karya, set `title`, `artist` (nama saya), `description`, dan `year` sesuai daftar ini: ... Atur posisinya rapi di tembok depan dan belakang.

B. Custom tembok motif Papua

Ubah `modules/walls.js` agar tembok memakai tekstur `public/img/papua.jpg` (bukan prosedural). Pakai `RepeatWrapping`, dan WAJIB prefix path dengan `import.meta.env.BASE_URL` supaya tetap jalan saat deploy ke GitHub Pages.

C. Perbaiki bug "karakter menembus patung" (*bug nyata di starter ini*)

Akar masalah: `checkCollision()` di `modules/movement.js` hanya mengecek `walls.children`. Patung dimuat async di `modules/statue.js` dan tak pernah didaftarkan sebagai penghalang.

Karakter bisa menembus patung di tengah ruangan. Tabrakan dihitung di `modules/movement.js` (`checkCollision`) yang hanya mengecek tembok. Tambahkan `collider` untuk patung: setelah patung selesai dimuat di `modules/statue.js`, buat `Box3` untuk patung dan sertakan dalam pengecekan tabrakan di `movement.js`. Tolong jaga agar perubahan minimal dan jelaskan langkahnya.

D. Minta penjelasan kode (belajar)

Jelaskan alur dari klik mouse sampai kamera zoom ke lukisan. File mana saja yang terlibat dan apa peran masing-masing?

Setelah agen mengubah kode: jalankan `npm run dev`, cek di browser, dan **review diff**-nya. Kalau salah, beri tahu agen apa yang keliru (iterasi).

7. Deploy ke GitHub Pages

7.1 Cara manual

1. **Buat repo** kosong di GitHub (mis. `nama/galeri-uas`).
2. Set base path di `vite.config.js` :

```
base: "/galeri-uas/", // samakan dengan NAMA REPO kalian
```

3. Build:

```
npm run build # menghasilkan folder dist/
```

4. Push isi `dist/` ke branch `gh-pages` :

```
cd dist
git init -q
git checkout -b gh-pages
git add -A
git commit -m "deploy"
git push -f https://github.com/NAMA/galeri-uas.git gh-pages
```

5. Di GitHub: **Settings** → **Pages** → **Build and deployment** → **Deploy from a branch** → **branch** `gh-pages` / **folder** / **(root)** → **Save**.
6. Tunggu ~1–2 menit. Situs live di `https://NAMA.github.io/galeri-uas/` . Kalau halaman putih, hard-refresh (Ctrl+Shift+R) dan pastikan `base` benar.

⚠️ Penyebab #1 layar putih di Pages = `base` salah, atau ada aset yang dimuat dengan path absolut tanpa `import.meta.env.BASE_URL` .

7.2 Cara via prompt (Claude Code / Codex)

```
Saya ingin deploy project Vite ini ke GitHub Pages di repo
https://github.com/NAMA/galeri-uas (branch gh-pages). Set base di vite.config.js
ke "/galeri-uas/", build, lalu push folder dist ke branch gh-pages. Setelah itu
beri tahu saya langkah mengaktifkan Pages di Settings.
```

8. Referensi

- **Repository asli:** theringsofsaturn / 3D-art-gallery-threejs — <https://github.com/theringsofsaturn/3D-art-gallery-threejs>
- **Tutorial freeCodeCamp (YouTube):** <https://www.youtube.com/watch?v=imqiYWidUIA&t=4859s>
- **Three.js docs:** <https://threejs.org/docs/>
- **Avaturn (foto → avatar):** <https://avaturn.me>
- **Mixamo (auto-rig & animasi):** <https://www.mixamo.com>
- **Blender (konversi FBX↔GLB):** <https://www.blender.org>
- **Kursus — Claude Code in Action** (Coursera/Anthropic).
- **Kursus — Introduction to OpenAI Codex** (Coursera/OpenAI).

Lisensi konten galeri asli: CC BY-NC-SA 4.0 (Emilian Kasemi). Cantumkan kredit ini pada laporan UAS kalian.